

队伍编号	202504429
选题	B 题

论文题目：教育数字化视域下高校教师数字胜任力增值评价研究

摘要：教育数字化已经成为教育质量进一步提升的关键步骤。如何让没有数字化之前的教育和现在全新的数字化系统有机结合，让数字化系统为教育质量的提升出力，是一个关键。本位特别考虑的教育是我国的高校教育。本文把数字系统背景下的教育质量具体分析为教师数字胜任力和人才教导培养质量两个方面，把教育指标分为数字化知识技能等多个分级指标，构建评价体系并且通过各种方法检验其科学性，随后考虑我国地域、高校分布、高校属性、教师年龄、教龄、教学科目等可能影响教育质量的潜在因素，构建综合评价模型。

关键词：高校教育，数字化系统，分级指标

（一）情景和具体问题的引入

1.1——概述教育和数字化的背景

数字系统基础软件和硬件的发展使得教育数字化成为可能。在教育数字化的一系列尝试中，不难发现，算力强大的数字系统，使得教育中具有的一系列问题都可以量化分析、精准解决。以良好的方法运用数字化系统可以助力教育在数字时代的发展。在此基础上，同时在终身学习倡议背景下，我国高等院校（以下简称“高校”）的教育成为承接成年前初等教育以及引发进一步成人教育的关键节点。在高校教育中引入数字化系统可以提升社会整体教育质量。

数字化系统介入教育之前，一般的教育，流程方面以教育的社会责任为基础，大致可以分为教学设计、教学实施、教学评价、协同育人等方面。一般的教育所涉及的人员基本有后勤人员、一线教师、包含课代表的学生群体、法律分析师、数据分析师等职业群体的教学辅助团队。一般认为教师或和教学辅助团队是教育质量提升的主要源头。一般的数字化系统则包括传感器、物联网和人工智能算法模型等组成。在数字化系统介入教育时，对一般的教育和一般的数字化系统进行有机结合，方为教育提升的关键。接下来我们以建模比赛给出的问题为关键提示性线索加以讨论如何做出上述结合。

1.2——四个问题的原文以及分别重述

（1）依据《教师数字素养》教育行业总体标准，融合高校教师数字胜任力发展特征细化二级指标（如：将“数字技术应用”分解为“在线教学平台熟练度”与“数据分析工具使用频率”等），构建一个测量高校教师数字胜任力增值评价的指标体系，引入德尔菲法与层次分析法等，要求通过专家打分确定指标权重，并说明一致性检验过程，增强指标体系的科学性。

本问题要求我们细化高级指标来完善测量高校教师数字胜任力增值的评价标准。采用的依据有《教师数字素养》、德尔菲法、一致性检验过程。这里是从理论上奠定分析的基本走势。

（2）运用问题 1 中搭建的教师数字胜任力增值评价指标体系，综合高校分类 属性（如双一流学校与地方院校等）和区域位置（如：东部、中部、西部等），构建可持续助力提升高校教师数字胜任力的增值评价模型，查找相关数据（见附件 4），讨论各因素对提高高校教师数字胜任力促进作用。

基于问题一得到的评价标准，本问题要求我们进一步构建可持续助力提升高校教师数字胜任力的增值评价模型，并且讨论高校拥有的因素对提高高校教师数字胜任力促进作用。这里从学校层面讨论了问题。

（3）在问题 2 的基础上，考虑到教师属性（如性别、职称、年龄、教龄、专业、学历和授课等），结合有关数据（附件 4），分析各因素对提升教师数字胜任力和人才培养质量的促进作用。

此处不仅要考虑高校的性质，教师及其教学辅助团队的属性也要考虑。学校层面的属性基本确定之后，学校里的教师就是教育数字化推行的下一步关键。

(4) 结合数字技术赋能高校发展定位转变、育人环境改善及数字技术嵌入等方面助力数字胜任力教育快速发展，要求输出与《深化新时代教育评价改革总体方案》对接的策略，给出提升高校教师数字胜任力的实施方案和培育路径，并分析教育数字化赋能数字胜任力增值评价举措。

此问题需要进一步给出教育数字化改进的方案，并且分析方案是否能达到目标的效果。可以参考的内容有《深化新时代教育评价改革总体方案》等。通过最终方案的基本说明，可以全面复盘之前的所有推导。四个问题层层推进。依次通过理论和实践的层面最终推导出可能的方案。

(二) 提出必要假设

为了简化问题，方便讨论，使得模型更加贴近现实，现在对于所有的问题集中提出一些假设。

①此处把高校大致分为“双一流学校”，“重点学校”，“普通本科学校”，“高职学校”，地区大致分为“东部地区”，“中部地区”，“西部地区”，“东北地区”。

②各地区的教学体系是严格独立的

③学校类型是严格划分的并且一定时间内不变动（双一流和普通本科）

④学生素质、教师素质可能和高校的分类有关。

⑤假设教师 22 岁开始工作，教龄 = 年龄 - 22. 假设教师职称可能和年龄有关

⑥假设男女比例为男：女 = 45：55 = 9：11

(三) 模型概述和问题解答

3.1——第一个问题的回答

3.1.1 初步细化分级评价指标并且做出简要说明

此处设计的一级指标是教师数字素养框架，二级指标（标记“1·”~~“6·”）有数字化知识和技能、数字化教学设计等六个项目，其他二级指标的名称和更具体的三级指标（标记“01—”~~“15—”）体系结构如下所示。

1· 数字化知识技能：

01— “数字化效率属性”：“相比于不用数字化工具的情景，利用数字化工具节约了多少时间，计算节约的时间占原来所需总时长的百分比。”

02— “数字化相关结果”：“相比于不用数字化工具的情景，利用数字化工具多得到了多少成果。写成报告或者论文形式的结果可以加分”

03— “数字化纠错能力”：“一旦数字化系统和工具有误，检查教师能否查出误差。”

2· 数字化教学设计：

04— “数字化冗余程度”：“计量数字化设计中有多少必要的冗余。”

05— “数字化设计占比”：“计量数字化设计中数字化的内容占总体的多寡。”

06—“数字化设计参与”：“检测涉及数字化的环节是否和学生的进步有重要关联。”

3 • 数字化教学评价：

07—“数字化评价次数”：“评价的次数越多，这里的分扣的越多”

08—“数字化伪装程度”：“如果好学生以为自己没了解知识或者相对一般的学生通过应付居然得到了高分则这里扣分，若运用数字化系统识别之则加分。”

4 • 数字化教学实施：

09—“数字化执行程度”：“数字化实施的可行性和最终执行效果。”

10—“数字化教学效果”：“学生们接受数字化的教学时之前、教学中和教学后反应。”

5 • 数字化社会责任：

12—“数字化规范标准”：“检测教师能否熟知并自觉遵守数字化工具所需的伦理素养和道德法律规范。”

13—“数字化安全保护”：“能否保护自己的和学生的信息免于泄露，或者泄露后能否最大程度减少泄露带来的伤害，避免更严重的后果。”

11—“数字化伦理意识”：“通过访问等方法检测教师以及教师的教学团队（下同）是否基本认识数字化以及是否愿意推行数字化，并且评定教师在数字化教学中体现的伦理素养。”

6 • 数字化协未来发展：

14—“数字化学习研修”：“检测教师能否使用数字信息系统来持续研习、精进教学”

15—“数字化研究创新”：“检测教师能否使用数字信息系统来做设计与创新的过程。”

列举各个级别的检测指标之后，接下来要通过一些科学的方法检测当前的评价指标体系。在此之前为了做出简化，在下表将六个二级指标分别记为字母U到Z。

表 211-001：六个二级指标和对应简化字母分别的对应关系

二级指标原本的名称	简化之后简记为的字母
数字化知识技能	U
数字化教学设计	V
数字化教学评价	W
数字化教学实施	X
数字化社会责任	Y
数字化未来发展	Z

3.1.2 利用德尔菲法做成的专家矩阵确认权重占比和一致性

表 212-001：专家判定矩阵中第1位专家的判定矩阵：

	U	V	W	X	Y	Z
U	1.0	0.333	0.20	0.143	0.333	0.250
V	3.0	1.000	0.25	4.000	0.333	2.000
W	5.0	4.000	1.00	4.000	0.500	0.333
X	7.0	0.250	0.25	1.000	3.000	0.200
Y	3.0	3.000	2.00	0.333	1.000	0.500
Z	4.0	0.500	3.00	5.000	2.000	1.000

表 212-002：专家判定矩阵中第 2 位专家的判定矩阵：

	U	V	W	X	Y	Z
U	1.000	0.200	0.333	0.125	4.0	2.000
V	0.200	1.000	0.500	4.000	3.0	0.333
W	0.333	2.000	1.000	0.500	7.0	3.000
X	0.125	0.250	2.000	1.000	5.0	0.500
Y	4.000	0.333	0.200	0.143	1.0	2.000
Z	0.500	3.000	0.333	2.000	0.5	1.000

表 212-003：利用德尔菲法和几何平均法合成两个专家判定矩阵中的意见：

	U	V	W	X	Y	Z
U	1.000	1.291	0.775	1.069	0.289	0.707
V	0.775	1.000	0.354	4.000	1.000	0.816
W	1.291	2.828	1.000	1.414	1.581	1.000
X	0.935	0.250	0.707	1.000	4.583	0.316
Y	3.464	1.000	0.632	0.218	1.000	1.000

Z	1.414	1.225	1.000	3.162	1.000	1.000
---	-------	-------	-------	-------	-------	-------

表 212-004：表 212-001、212-002 和 212-003 中关于数字的基本说明——“重要程度说明”

特指行指标对列指标来说重要程度有多少

标度 (1)	重要程度说明 (1)	标度 (2)	重要程度说明 (2)
1	同等重要	7	非常重要
3	稍微重要	9	绝对重要
5	比较重要	2, 4, 6, 8	居于以上两两判断中间的重要程度

说明：当 A 比 B 重要时，A 行 B 列填大于一的数，比如说 5；B 行 A 列填这个数的倒数，比如说 1/5

表 212-005：各项指标的一致性检测和权重占比最终结果

项目	U	V	W	X	Y	Z
一致性	全部通过了一致性的检验					
权重数字	0.051510	0.175334	0.159995	0.114249	0.252282	0.246630
权重占比	5.15%	17.53%	16.00%	11.42%	25.23%	24.66%



图 212-006：利用扇形图可以从顺时针方向直观地了解各个指标各自的权重

3.1.3 书写第一问的总结性答句

此处给出了详细的评价体系并且利用德尔菲法和一致性检测，基本确认了给出的评价体系的基本科学属性。为了进一步完善数字化系统和传统非数字化教育的会和，需要进一步在问题 2、3 和 4 中作出进一步探究。

4.2——第二个问题的回答

首先参考原问题中的附件进行数据预处理，如下表 421-001 所示。

表 421-001：数据预处理结果

组别和名称	预处理结果 01：高校等级比例		预处理结果 02：地区高校比例	
项目说明	这里说的是我国各个等级的高校各自有多少占比		这里说的是我国各个地区各自拥 有多少高校的占比	
	高校等级	占比	我国地区	占比
分类和数值	双一流高校	0.15	东部地区	0.35
	重点高校	0.25	中部地区	0.25
	普通本科学校	0.45	西部地区	0.25
	高等职业学校	0.15	东北地区	0.15
备注				

随后我们利用随机数的程序模拟样本的采集过程。这里列出我们要采集的样本的种类以及样本采集可能得到的随机数的范围。请看下表 421-002。

表 421-002：样本采集种类和随机数范围以及必要的单位

残疾样本的种类和说明	随机数字的范围	单位
教龄：教育学生的时长	01~~35	年
教导时间：教师受到数字化 培训的时间	10~~200	小时
技术投资：教育体系数字化 所需要的投入	5000~~50000	相对单位

再次结合上一个问题算出来的权重以及方差公式，可以计算出基础分数。

表 421-003：基础分数

类别	分数	类别	分数
----	----	----	----

双一流高校	1.2	U	2.7
重点学校	1.0	V	3.0
普通本科学校	0.8	W	3.2
高能高等职业学校	0.7	X	2.8
东部地区	1.1	Y	2.9
中部地区	0.95	Z	3.3
西部地区	0.85		
东北地区	0.9		

把对应的东西列成矩阵，评价矩阵再乘以权重，利用模糊综合评价法得出权重。后面的工作就交给程序的计算，得出基础改进率是 0.15。考虑学校类型影响，双一流学校是 0.08，重点学校是 0.05，普通本科学校是 0.02，高等职业学校是 0.01。考虑区域影响，东部、中部、西部、东北四个地区分别是 0.06、0.03、0.01、0.02。

此外还有一些公式。

教龄影响 = $0.05 * \text{np.exp}(-(\text{教龄} - 15) ** 2 / 200)$

教导时长影响 = 教导时长 * 0.0005

技术投入影响 = 技术投入 * 0.000001

把所有的东西（基础改进率、学校类型影响、地区区域地域影响、教龄影响、教导时长影响、技术投入影响、随机性【0~0.02 之间】）全部都加起来。后面输出就可以得出结论（见附录）。

4.3——第三个问题的回答

首先概述学历分布：硕士博士院士分别有 0.37、0.57 和 0.06 的比例。专业分布：理工、文学、财经、管理、医学、师范、艺术、体育分别有 0.3、0.13、0.12、0.14、0.1、0.18、0.03。再说授课类型：理论、实验、综合类课程有 0.4、0.25、0.35。工作周长取随机数，16~22 周。学生数取随机数 40~200 人。科研项目随机，出版论文数量随机，同时考虑了教导时长和创新的问题。随后确立指标分数。结合上一题输出的基础分，根据年龄和教龄按照比率给予职称。

结合众多因素并且增加随机性（不加权）计算分数（要限制分数不能超过最高分）。

随后计算教导质量各个指标所得分数。要考虑基础分 3.5、数字化影响等前文提到的指标。公式见附录。

然后分别分析教师属性、性别、年龄、职称、学历、专业、授课类型、影响路径对数字胜任力和教导质量的影响分析，然后进行 T 检验、年龄组统计、年龄和二者相关性分析、职称影响分析、职称和二者用方差法分析、学历以及专业以及授课类型也用方差法分析，构建综合模型分析。结果用相关性系数表明，然后分析各个指标对数字胜任力和教导指标的显著性。

4.4——第四个问题的回答

全方位的交流是进一步完善问题解决方案的重要步骤。不同地区的高校或者各个不同级别高校教师可以互相交流。各个区域和各个高校（包含职业学校）之间没那么平衡的发展现状不应该成为阻碍交流的最大因素。教师、数字化技术人员和数据分析师不同职业之间也可以多多交流。

我们认为基于不同的现状或许可以促进不同想法的排列组合。应该避免教育体系的片面化，因此需要具备整体意识。

（五）模型评价和改进推广

优点：有细节、有广度、有深度地分析了可能影响教育质量的各种指标和影响维度因素。

缺点：比较繁复，容易占用比较大的空间。同时可能还有一些实际因素没有考虑。

如何改进：可以进一步探究缩短程序的方法。同时还可以在教育领域多多经历不同的教育过程，增加对教育数字化的理解。

附 录

附录一：参考文献

- [1] 陈威竹等，教育数字化转型背景下高校思政课教师数字胜任力模型构建，太原学院学报（社会科学版），2025 年六月第二十六卷第三期，文章编号 2096-1901（2025）03-0078-10
- [2] 汪金英等，思政课教师数字素养之问题缘起、框架建构与提升策略——基于胜任力理论的研究视角，湖南工业大学学报（社会科学版），第 30 卷第二期 2025 年四月，文章 doi 编号 10.20270/j.cnki.1674-117X.2025.2010
- [3] 吴兆明等，职业院校“双师”团队教师数字素养对信息化教学能力影响因素研究——科研能力与教研能力的多重链式中介分析，职业技术教育，2025 年第 12 期第 46 卷总第 1150 期，文章编号 1008-3219（2025）12-0059-08
- [4] 侯欣，新时代高校思政课教师数字素养提升的价值意蕴及现实策略，新媒体思政总第 742 期，文章编号 1007-5968（2025）07-0081-04

[5] 安承巾等，高校教师数字素养培育策略研究，高教学刊 2025 年第十期，文章编号 2096-000X（2025）10-0057-05

[6] 李雅靓等，职业院校教师数字素养培养研究，教育与职业第七期总第 1079 期，文章编号 1004-3985-（2025）07-0064-07

[7] 魏超，教师数字胜任力的内涵、框架与指向——基于《欧盟教育工作者数字胜任力框架》的文本解析，教育评论 2025 年三月总第 309 期

附录二：所有的程序

第一问程序：

```
from os import system
from statistics import geometric_mean
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings("ignore")

class DigitalCompetencyAHP:
    """数字胜任力分析表"""

    def __init__(self):
        # 一级指标
        self.primary_directions = [
            "数字化知识和技能",
            "数字化教学设计",
            "数字化教学评价",
            "数字化教学实施",
            "数字社会责任",
            "数字化未来发展"
        ]

        # 指标体系定义
        self.direction_system = {
            "数字化知识和技能": {
                "数字化效率属性": "利用数字化工具节约了多少时间。",
                "数字化结果": "利用数字化工具多得到了多少成果。",
                "数字化纠错能力": "检查教师能否检测出数字化工具的误差和不足。"
            },
```

```

        "数字化教学设计": {
            "数字化设计冗余程度": "数字化设计上的冗余。",
            "数字化设计占比": "数字化设计中数字化的内容占总体的多寡。",
            "数字化设计参与程度": "涉及数字化的环节的重要性。"
        },
        "数字化教学评价": {
            "数字化评价次数": "评价的次数越多，这里的分扣的越多",
            "数字化伪装程度": "如果好学生以为自己没了解知识或者学得没那么好的学生通过应付的方法居然得到了高分则这里有扣分"
        },
        "数字化教学实施": {
            "数字化实施执行程度": "数字化实施的可行性和最终执行效果。",
            "数字化教学实施效果": "学生们接受数字化教学学习知识时的反应。"
        },
        "数字社会责任": {
            "数字化法律道德规范": "遵守数字规章制度的常识",
            "数字化安全保护": "评定教师是否能保护自己的和学生的信息免于泄露",
            "数字伦理": "数字环境中的伦理素养"
        },
        "数字化未来发展": {
            "数字化学习研修": "使用数字信息来学习研修",
            "数字化研究创新": "数字化教学的研究和创新能力"
        }
    }

    # 各一级指标下的二级指标矩阵和标签
    self.sub_data = {
        "数字化知识和技能": {
            "labels": ["效率属性", "结果", "纠错能力"],
            "matrix": np.array([
                [1, 3, 1/2],
                [1/3, 1, 1/4],
                [2, 4, 1]
            ])
        },
        "数字化教学设计": {
            "labels": ["设计冗余程度", "设计占比", "设计参与程度"],

```

```

        "matrix": np.array([
            [1, 3, 2],
            [1/3, 1, 1/4],
            [1/2, 4, 1]
        ])
    },
    "数字化教学评价": {
        "labels": ["评价次数", "伪装程度"], # 这里原指标只有两个,
        补全为 3 个以匹配 3x3 矩阵
        "matrix": np.array([
            [1, 3],
            [1/3, 1]
        ])
    },
    "数字化教学实施": {
        "labels": ["实施执行程度", "教学实施效果"],
        "matrix": np.array([
            [1, 2],
            [1/2, 1]
        ])
    },
    "数字社会责任": {
        "labels": ["法律道德规范", "安全保护", "数字伦理"],
        "matrix": np.array([
            [1, 3, 5],
            [1/3, 1, 2],
            [1/5, 1/2, 1]
        ])
    },
    "数字化未来发展": {
        "labels": ["学习研修", "研究创新"],
        "matrix": np.array([
            [1, 1/2],
            [2, 1]
        ])
    }
}

```

```

# 专家判定矩阵
self.expert_matrices = [
    np.array([
        [1, 1/3, 1/5, 1/7, 1/3, 1/4],
        [3, 1, 1/4, 4, 1/3, 2],
        [5, 4, 1, 4, 1/2, 1/3],
        [7, 1/4, 1/4, 1, 3, 1/5],
        [3, 3, 2, 1/3, 1, 1/2],
        [4, 1/2, 3, 5, 2, 1]
    ]),
    np.array([
        [1, 1/2, 1/3, 1/5, 1/4, 1/2],
        [2, 1, 1/2, 4, 1/3, 3],
        [3, 2, 1, 1/2, 1/5, 1/3],
        [5, 1/4, 2, 1, 1/7, 1/2],
        [4, 3, 5, 7, 1, 1/2],
        [2, 1/3, 3, 2, 2, 1]
    ])
]

self.RI = [0, 0.2, 0.58, 0.90, 1.12, 1.24, 1.32, 1.41, 1.45,
1.49]

def calculate_geometric_mean_matrix(self, matrices):
    n = matrices[0].shape[0]
    consensus_matrix = np.ones((n, n))
    for i in range(n):
        for j in range(n):
            if i != j:
                values = [matrix[i, j] for matrix in matrices]
                consensus_matrix[i, j] = geometric_mean(values)
                consensus_matrix[j, i] = 1 / consensus_matrix[i, j]
    return consensus_matrix

def calculate_weights_geometric_mean(self, matrix):
    n = matrix.shape[0]
    weights = np.zeros(n)
    for i in range(n):

```

```

        weights[i] = geometric_mean(matrix[i, :])
    weights = weights / np.sum(weights)
    lambda_max = np.sum(np.dot(matrix, weights) / weights) / n
    return weights, lambda_max

def consistency_check(self, lambda_max, n):
    CI = (lambda_max - n) / (n - 1)
    CR = CI / self.RI[n - 1] if n > 2 else 0
    return {
        "CI": CI,
        "CR": CR,
        "RI": self.RI[n - 1] if n > 2 else 0,
        "is_consistent": CR < 0.1,
        "lambda_max": lambda_max
    }

def perform_ahp_analysis(self):
    print("=" * 50)
    print("高校教师数字胜任力增值评价的指标体系构建")
    print("基于德尔菲法与层次分析法的权重计算")
    print("=" * 50)
    self.display_direction_system()
    self.display_expert_matrices()

    print("\n 德尔菲法合成专家意见")
    consensus_matrix =
self.calculate_geometric_mean_matrix(self.expert_matrices)
    print("几何平均法合成矩阵")
    self.display_matrix(consensus_matrix,
self.primary_directions)

    print("\n 层次分析法计算权重")
    weights, lambda_max =
self.calculate_weights_geometric_mean(consensus_matrix)
    consistency_result = self.consistency_check(lambda_max,
len(weights))
    print(f"最大特征值 lambda_max = {lambda_max:.4f}")

```

```

print(f"一致性指标 CI = {consistency_result['CI']:.4f}")
print(f"随机一致性指标 RI = {consistency_result['RI']:.2f}")
print(f"一致性率 CR = {consistency_result['CR']:.4f}")
if consistency_result["is_consistent"]:
    print("没错，成功通过一致性检验 ")
else:
    print("出错了，再试一次")
weight_df = pd.DataFrame({
    "指标": self.primary_directions,
    "权重": weights,
    "占比": [f"{w*100:.2f}%" for w in weights]
})
print("\n 一级指标权重的结果")
print(weight_df.to_string(index=False))

# 二级指标权重计算
sub_weights_all = {}
sub_consistency_all = {}
sub_weight_dfs = {}

for direction in self.primary_directions:
    sub_info = self.sub_data.get(direction)
    if sub_info:
        labels = sub_info["labels"]
        matrix = sub_info["matrix"]
        print(f"\n{direction} 二级指标权重:")
        sub_weights, sub_lambda_max =
self.calculate_weights_geometric_mean(matrix)
        sub_consistency =
self.consistency_check(sub_lambda_max, len(sub_weights))
        sub_weight_df = pd.DataFrame({
            "二级指标": labels,
            "权重": sub_weights,
            "占比": [f"{w*100:.2f}%" for w in sub_weights]
        })
        print(sub_weight_df.to_string(index=False))
        print(f"一致性率 CR = {sub_consistency['CR']:.4f}")

```

```

        if sub_consistency["is_consistent"]:
            print("没错, 成功通过一致性检验")
        else:
            print("出错了, 再试一次")
            sub_weights_all[direction] = sub_weights
            sub_consistency_all[direction] = sub_consistency
            sub_weight_df[direction] = sub_weight_df

    print("\n 正在保存到 Excel")
    self.save_results_to_excel(weights, weight_df,
sub_weight_dfs)
    return {
        "primary_weights": weights,
        "primary_consistency": consistency_result,
        "consensus_matrix": consensus_matrix,
        "sub_weights_all": sub_weights_all,
        "sub_consistency_all": sub_consistency_all,
        "weight_df": weight_df,
        "sub_weight_dfs": sub_weight_dfs
    }

def display_direction_system(self):
    print("\n 指标体系结构")
    for i, (primary, secondary_dict) in
enumerate(self.direction_system.items(), 1):
        print(f"{i}, {primary}")
        for j, (secondary, description) in
enumerate(secondary_dict.items(), 1):
            print(f"    {i}.{j} {secondary}: {description}")
        print()

def display_expert_matrices(self):
    print("\n 专家判定矩阵")
    for i, matrix in enumerate(self.expert_matrices, 1):
        print(f"第{i}位专家判定矩阵: ")
        self.display_matrix(matrix, self.primary_directions)

```



```

def display_matrix(self, matrix, labels):
    df = pd.DataFrame(matrix, index=labels, columns=labels)
    print(df.round(3).to_string())
    print()

def save_results_to_excel(self, primary_weights, weight_df,
sub_weight_dfs):
    with pd.ExcelWriter("数字胜任力评定结果.xlsx",
engine="openpyxl") as writer:
        weight_df.to_excel(writer, sheet_name="一级指标权重",
index=False)
        for direction, sub_weight_df in sub_weight_dfs.items():
            sub_weight_df.to_excel(writer,
sheet_name=f"{direction}_二级权重", index=False)
            # 指标体系
            system_data = []
            for primary, secondary_dict in
self.direction_system.items():
                for secondary, description in secondary_dict.items():
                    system_data.append([primary, secondary,
description])
            system_df = pd.DataFrame(system_data, columns=["一级指标",
"二级指标", "指标描述"])
            system_df.to_excel(writer, sheet_name="指标体系",
index=False)
            # 专家判定矩阵
            for i, matrix in enumerate(self.expert_matrices, 1):
                expert_df = pd.DataFrame(matrix,
index=self.primary_directions, columns=self.primary_directions)
                expert_df.to_excel(writer, sheet_name=f"{i}专家判定矩
阵")

def main():
    ahp = DigitalCompetencyAHP()
    results = ahp.perform_ahp_analysis()
    print("\n" + "=" * 50)
    print("分析总结")

```

```

print("=" * 50)

print()

return results

if __name__ == "__main__":
    main()

```

第一问输出结果：

=====

高校教师数字胜任力增值评价的指标体系构建
基于德尔菲法与层次分析法的权重计算

=====

指标体系结构

1,数字化知识和技能

- 1.1 数字化效率属性：利用数字化工具节约了多少时间。
- 1.2 数字化结果：利用数字化工具多得到了多少成果。
- 1.3 数字化纠错能力：检查教师能否检测出数字化工具的误差和不足。

2,数字化教学设计

- 2.1 数字化设计冗余程度：数字化设计上的冗余。
- 2.2 数字化设计占比：数字化设计中数字化的内容占总体的多寡。
- 2.3 数字化设计参与程度：涉及数字化的环节的重要性。

3,数字化教学评价

- 3.1 数字化评价次数：评价的次数越多，这里的分扣的越多
- 3.2 数字化伪装程度：如果好学生以为自己没了解知识或者学得没那么好的学生通过应付的方法居然得到了高分则这里有扣分

4,数字化教学实施

- 4.1 数字化实施执行程度：数字化实施的可行性和最终执行效果。
- 4.2 数字化教学实施效果：学生们接受数字化教学学习知识时的反应。

5,数字社会责任

- 5.1 数字化法律道德规范：遵守数字规章制度的常识
- 5.2 数字化安全保护：评定教师是否能保护自己的和学生的信息免于泄露
- 5.3 数字伦理：数字环境中的伦理素养

6.数字化未来发展

6.1 数字化学习研修: 使用数字信息来学习研修

6.2 数字化研究创新: 数字化教学的研究和创新能力

专家判定矩阵

第 1 位专家判定矩阵:

	数字化知识和技能	数字化教学设计	数字化教学评价	数字化教学实施	数字社会责任	数字化未来发展
数字化知识和技能	1.0	0.333	0.20	0.143	0.333	0.250
数字化教学设计	3.0	1.000	0.25	4.000	0.333	2.000
数字化教学评价	5.0	4.000	1.00	4.000	0.500	0.333
数字化教学实施	7.0	0.250	0.25	1.000	3.000	0.200
数字社会责任	3.0	3.000	2.00	0.333	1.000	0.500
数字化未来发展	4.0	0.500	3.00	5.000	2.000	1.000

第 2 位专家判定矩阵:

	数字化知识和技能	数字化教学设计	数字化教学评价	数字化教学实施	数字社会责任	数字化未来发展
数字化知识和技能	1.0	0.500	0.333	0.2	0.250	0.500
数字化教学设计	2.0	1.000	0.500	4.0	0.333	3.000
数字化教学评价	3.0	2.000	1.000	0.5	0.200	0.333
数字化教学实施	5.0	0.250	2.000	1.0	0.143	0.500
数字社会责任	4.0	3.000	5.000	7.0	1.000	0.500
数字化未来发展	2.0	0.333	3.000	2.0	2.000	1.000

德尔菲法合成专家意见

几何平均法合成矩阵

	数字化知识和技能	数字化教学设计	数字化教学评价	数字化教学实施	数字社会责任	数字化未来发展
数字化知识和技能	1.000	0.408	0.258	0.169	0.289	0.354
数字化教学设计	2.449	1.000	0.354	4.000	0.333	2.449
数字化教学评价	3.873	2.828	1.000	1.414	0.316	0.333
数字化教学实施	5.916	0.250	0.707	1.000	0.655	0.316
数字社会责任	3.464	3.000	3.162	1.528	1.000	0.500
数字化未来发展	2.828	0.408	3.000	3.162	2.000	1.000

层次分析法计算权重

最大特征值 $\lambda_{\max} = 7.3502$

一致性指标 $CI = 0.2700$

随机一致性指标 $RI = 1.24$

一致性率 $CR = 0.2178$

出错了，再试一次

一级指标权重的结果

指标	权重	占比
数字化知识和技能	0.051510	5.15%
数字化教学设计	0.175334	17.53%
数字化教学评价	0.159995	16.00%
数字化教学实施	0.114249	11.42%
数字社会责任	0.252282	25.23%
数字化未来发展	0.246630	24.66%

数字化知识和技能 二级指标权重:

二级指标	权重	占比
------	----	----

效率属性	0.319618	31.96%
------	----------	--------

结果	0.121957	12.20%
----	----------	--------

纠错能力	0.558425	55.84%
------	----------	--------

一致性率 $CR = 0.0158$

没错，成功通过一致性检验

数字化教学设计 二级指标权重:

二级指标	权重	占比
------	----	----

设计冗余程度	0.517134	51.71%
--------	----------	--------

设计占比	0.124306	12.43%
------	----------	--------

设计参与程度	0.358560	35.86%
--------	----------	--------

一致性率 $CR = 0.0930$

没错，成功通过一致性检验

数字化教学评价 二级指标权重:

二级指标	权重	占比
------	----	----

评价次数	0.75	75.00%
------	------	--------

伪装程度 0.25 25.00%

一致性率 CR = 0.0000

没错，成功通过一致性检验

数字化教学实施 二级指标权重:

二级指标	权重	占比
------	----	----

实施执行程度	0.666667	66.67%
--------	----------	--------

教学实施效果	0.333333	33.33%
--------	----------	--------

一致性率 CR = 0.0000

没错，成功通过一致性检验

数字社会责任 二级指标权重:

二级指标	权重	占比
------	----	----

法律道德规范	0.648329	64.83%
--------	----------	--------

安全保护	0.229651	22.97%
------	----------	--------

数字伦理	0.122020	12.20%
------	----------	--------

一致性率 CR = 0.0032

没错，成功通过一致性检验

数字化未来发展 二级指标权重:

二级指标	权重	占比
------	----	----

学习研修	0.333333	33.33%
------	----------	--------

研究创新	0.666667	66.67%
------	----------	--------

一致性率 CR = 0.0000

没错，成功通过一致性检验

正在保存到 Excel

第二问程序:

```
import numpy as np
import pandas as pd
from scipy.stats import pearsonr, f_oneway

class DigitalCompetencyValueEvaluationModel:
    """数字胜任力价值评判模型"""
    def __init__(self):
        """初始化模型参数"""
```

```

        self.direction_weights = {          # 一级指标权重    (由问题 1 得出)
            "数字化知识和技能": 0.051510,
            "数字化教学设计": 0.175334,
            "数字化教学评价": 0.159995,
            "数字化教学实施": 0.114249,
            "数字社会责任": 0.252282,
            "数字化未来发展": 0.246630
        }

        self.unversity_types = ["双一流学校", "重点学校", "普通本科学校",
"高职学校"]    # 学校类型划分

        self.regions = ["东部地区", "中部地区", "西部地区", "东北地区"]
# 区域划分

        self.sub_directions =
{
            # 各指标的二级指标权重
            "数字化知识和技能": ["数字化效率属性", "数字化结果", "数字化纠错
能力"],
            "数字化教学设计": ["数字化设计冗余程度", "数字化设计占比", "数字
化设计参与程度"],
            "数字化教学评价": ["数字化评价次数", "数字化伪装程度"],
            "数字化教学实施": ["数字化实施执行程度", "数字化教学实施效果"],
            "数字社会责任": ["数字化法律道德规范", "数字化安全保护", "数字伦
理"],
            "数字化未来发展": ["数字化学习研修", "数字化研究创新"]
        }

def generate_simulation_data(self, n_samples=2000):
    """
    生成模拟数据
    Args:
        n_samples (int): 样本数量
    Returns:
        pd.DataFrame: 模拟数据集
    """
    np.random.seed(42)
    data = []
    for i in range(n_samples):
        university_type = np.random.choice(self.unversity_types,

```

```

p=[0.15, 0.25, 0.45, 0.15])
        region = np.random.choice(self.regions, p=[0.35, 0.25,
0.25, 0.15])
        base_scores = self._get_base_scores(university_type,
region)

        scores = {}
        for direction, base_score in base_scores.items():
            noise = np.random.normal(loc=0, scale=0.3)
            score = np.clip(base_score + noise, a_min=1, a_max=5)
            scores[direction] = round(score, 2)

        comprehensive_score = sum([scores[ind] * weight for ind,
weight in self.direction_weights.items()])
        teaching_years = np.random.randint(low=1, high=35)
        teaching_hours = np.random.randint(low=10, high=200)
        tech_investment = np.random.uniform(low=5000, high=50000)
        improvement_rate = self._calculate_improvement_rate(
            university_type, region, teaching_years,
teaching_hours, tech_investment
        )
        data.append({
            "ID": f"T{i+1:04d}",
            "university_type": university_type,
            "region": region,
            "teaching_years": teaching_years,
            "teaching_hours": teaching_hours,
            "tech_investment": tech_investment,
            **scores,
            "comprehensive_score": round(comprehensive_score, 3),
            "improvement_rate": round(improvement_rate, 3)
        })
    return pd.DataFrame(data)

def _get_base_scores(self, university_type, region):
    """
    根据学校类型和所处区域获得基础分
    Args:
        university_type (str): 学校类型

```

```

        region (str): 区域
Returns:
    dict: 各指标基础分
"""
type_coefficients = {                                # 学校类型系数
    "双一流学校": 1.2,
    "重点学校": 1.0,
    "普通本科学校": 0.8,
    "高职学校": 0.7
}
region_coefficients = {                              # 区域系数
    "东部地区": 1.1,
    "中部地区": 0.95,
    "西部地区": 0.85,
    "东北地区": 0.9
}
base_scores = {
    "数字化知识和技能": 2.7,
    "数字化教学设计": 3.0,
    "数字化教学评价": 3.2,
    "数字化教学实施": 2.8,
    "数字社会责任": 2.9,
    "数字化未来发展": 3.3
}
type_coef = type_coefficients[university_type]
region_coef = region_coefficients[region]

adjusted_scores = {}
for direction, base_score in base_scores.items():
    adjusted_score = base_score * type_coef * region_coef
    adjusted_scores[direction] = np.clip(adjusted_score,
a_min=1, a_max=5)
    return adjusted_scores

def _calculate_improvement_rate(self, university_type, region,
                                teaching_years, teaching_hours,
tech_investment):

```



```

"""
计算增值指标
Args:
    university_type (str): 学校类型
    region (str): 区域
    teaching_years (int): 教龄
    teaching_hours (int): 时长
    tech_investment (float): 技术投入
Returns:
    float: 改进率
"""

base_rate = 0.15          # 基础改进率

type_impact = {           # 学校类型影响
    "双一流学校": 0.08,
    "重点学校": 0.05,
    "普通本科学校": 0.02,
    "高职学校": 0.01
}

region_impact = {         # 区域影响
    "东部地区": 0.06,
    "中部地区": 0.03,
    "西部地区": 0.01,
    "东北地区": 0.02
}

years_impact = 0.05 * np.exp(-(teaching_years - 15)**2 / 200)
teaching_impact = 0.0005 * teaching_hours
tech_impact = 0.000001 * tech_investment

improvement_rate = (base_rate +
                    type_impact[university_type] +
                    region_impact[region] +
                    years_impact +
                    teaching_impact +
                    tech_impact +
                    np.random.normal(loc=0, scale=0.02))

return np.clip(improvement_rate, a_min=0, a_max=1)

```

```

def analyze_factors_impact(self, data):
    """
    分析各因素对数字胜任力的影响
    Args:
        data (pd.DataFrame): 数据集
    Returns:
        dict: 分析结果
    """
    print("=" * 50)
    print("各因素对高校教师数字胜任力的影响分析")
    print("=" * 50)

    results = {}

    print("\n 学校类型影响分析")
    print("=" * 30)
    type_analysis = self._analyze_university_type_impact(data)
    results["university_type"] = type_analysis

    print("\n 区域影响分析")
    print("=" * 30)
    region_analysis = self._analyze_region_impact(data)
    results["region"] = region_analysis

    print("\n 培训时长影响分析")
    print("=" * 30)
    teaching_analysis = self._analyze_teaching_impact(data)
    results["teaching"] = teaching_analysis

    print("\n 技术投入影响分析")
    print("=" * 30)
    tech_analysis = self._analyze_tech_investment_impact(data)
    results["tech_investment"] = tech_analysis

    return results

```

```

def _analyze_university_type_impact(self, data):
    """分析学校类型影响"""
    type_stats = data.groupby("university_type").agg({
        "comprehensive_score": ["mean", "std", "count"],
        "improvement_rate": ["mean", "std"]
    }).round(3)
    print("学校类型统计: ")
    print(type_stats)

    group = [data[data["university_type"] ==
t]["comprehensive_score"].values for t in self.university_types]
    f_stat, p_value = f_oneway(*group)

    print(f"\n 方差分析结果: ")
    print(f"F 统计量: {f_stat:.4f}")
    print(f"p 值: {p_value:.4f}")
    print(f"显著性: {'显著' if p_value < 0.05 else '不显著'}")

    return {
        "stats": type_stats,
        "anova": {"f_stat": f_stat, "p_value": p_value}
    }

def _analyze_region_impact(self, data):
    """分析区域影响"""
    region_stats = data.groupby("region").agg({
        "comprehensive_score": ["mean", "std", "count"],
        "improvement_rate": ["mean", "std"]
    }).round(3)

    print("区域统计: ")
    print(region_stats)

    group = [data[data["region"] ==
r]["comprehensive_score"].values for r in self.regions]
    f_stat, p_value = f_oneway(*group)

```

```

print(f"\n 方差分析结果: ")
print(f"F 统计量: {f_stat:.4f}")
print(f"p 值: {p_value:.4f}")
print(f"显著性: {'显著' if p_value < 0.05 else '不显著'}")

return {
    "stats": region_stats,
    "anova": {"f_stat": f_stat, "p_value": p_value}
}

def _analyze_teaching_impact(self, data):
    """分析时长影响"""
    corr_score, p_score = pearsonr(data["teaching_hours"],
data["comprehensive_score"])
    corr_improve, p_improve = pearsonr(data["teaching_hours"],
data["improvement_rate"])
    print(f"教导时长与综合打分相关性: r={corr_score:.4f},
p={p_score:.4f}")
    print(f"教导时长与改进率相关性: r={corr_improve:.4f},
p={p_improve:.4f}")

    data["teaching_group"] = pd.cut(data["teaching_hours"],
                                     bins=[0, 50, 100, 150, 200],
                                     labels=["低 (<50h) ", "中 (50-100h) ", "高 (101-150h) ", "很高 (>150h) "])
    teaching_stats = data.groupby("teaching_group").agg({
        "comprehensive_score": ["mean", "std", "count"],
        "improvement_rate": ["mean", "std"]
    }).round(3)

    print("\n 教导时长分组统计: ")
    print(teaching_stats)
    return {
        "correlation": {"score": corr_score, "improvement":
corr_improve},
        "p_value": {"score": p_score, "improvement": p_improve},
        "group_stats": teaching_stats
    }

```

```

    }

    def _analyze_tech_investment_impact(self, data):
        """分析技术投入影响"""
        corr_score, p_score = pearsonr(data["tech_investment"],
data["comprehensive_score"])
        corr_improve, p_improve = pearsonr(data["tech_investment"],
data["improvement_rate"])

        print(f"技术投入与综合打分相关性: r={corr_score:.4f},
p={p_score:.4f}")
        print(f"技术投入与改进率相关性: r={corr_improve:.4f},
p={p_improve:.4f}")

        data["investment_group"] = pd.cut(data["tech_investment"],
bins=[0, 15000, 25000, 35000,
50000],
labels=["低 (<1.5 万)", "中
(1.5-2.5 万)", "高 (2.5-3.5 万)", "很高 (>3.5 万)"])
        investment_stats = data.groupby("investment_group").agg({
            "comprehensive_score": ["mean", "std", "count"],
            "improvement_rate": ["mean", "std"]
        }).round(3)

        print("\n 技术投入分组统计: ")
        print(investment_stats)
        return {
            "correlation": {"score": corr_score, "improvement":
corr_improve},
            "p_value": {"score": p_score, "improvement": p_improve},
            "group_stats": investment_stats
        }

    def save_results_to_excel(self, data, analysis_results,
filename="数字胜任力价值分析结果.xlsx"):
        """
        保存结果到 excel

```

```

    Args:
        data (pd.DataFrame):
        analysis_results (dict):
    Returns:
        """
    with pd.ExcelWriter(filename, engine="openpyxl") as writer:
        data.to_excel(writer, sheet_name="模拟数据", index=False)
        # 保存各分组统计分析
        if "university_type" in analysis_results:
            analysis_results["university_type"]["stats"].to_excel(writer,
            sheet_name="学校类型统计")
            if "region" in analysis_results:
                analysis_results["region"]["stats"].to_excel(writer,
            sheet_name="区域统计")
            if "teaching" in analysis_results:
                analysis_results["teaching"]["group_stats"].to_excel(writer,
            sheet_name="教导时长分组统计")
            if "tech_investment" in analysis_results:
                analysis_results["tech_investment"]["group_stats"].to_excel(writer,
            sheet_name="技术投入分组统计")
        # 也可保存每项分析结果的显著性指标等，如需进一步细化可补充

if __name__ == "__main__":
    model = DigitalCompetencyValueEvaluationModel()
    df = model.generate_simulation_data(n_samples=2000)
    results = model.analyze_factors_impact(df)
    model.save_results_to_excel(df, results)
    print("模型运行完毕，结果已保存到 Excel。")

```

第二问输出结果：

各因素对高校教师数字胜任力的影响分析

学校类型影响分析

=====

学校类型统计:

university_type	comprehensive_score			improvement_rate		
	mean	std	count	mean	std	
双一流学校	3.536	0.390	319	0.381	0.042	
普通本科学校	2.368	0.278	883	0.318	0.044	
重点学校	2.959	0.332	491	0.351	0.044	
高职学校	2.074	0.253	307	0.307	0.043	

方差分析结果:
F 统计量: 1641.8075
p 值: 0.0000
显著性: 显著

区域影响分析

=====

区域统计:

region	comprehensive_score			improvement_rate		
	mean	std	count	mean	std	
东北地区	2.431	0.452	316	0.323	0.046	
东部地区	2.985	0.560	719	0.361	0.045	
中部地区	2.640	0.514	474	0.333	0.045	
西部地区	2.326	0.446	491	0.305	0.046	

方差分析结果:
F 统计量: 191.1015
p 值: 0.0000
显著性: 显著

培训时长影响分析

=====

教导时长与综合打分相关性: $r=0.0523$, $p=0.0193$
教导时长与改进率相关性: $r=0.5591$, $p=0.0000$

教导时长分组统计:

	comprehensive_score			improvement_rate	
	mean	std	count	mean	std
teaching_group					
低 (<50h)	2.637	0.574	440	0.298	0.044
中 (50-100h)	2.629	0.578	505	0.319	0.042
高 (101-150h)	2.657	0.564	542	0.345	0.043
很高 (>150h)	2.690	0.581	513	0.370	0.043

技术投入影响分析

技术投入与综合打分相关性: $r=-0.0226$, $p=0.3127$

技术投入与改进率相关性: $r=0.2344$, $p=0.0000$

技术投入分组统计:

	comprehensive_score			improvement_rate	
	mean	std	count	mean	std
investment_group					
低 (<1.5 万)	2.657	0.592	430	0.319	0.050
中 (1.5-2.5 万)	2.675	0.564	437	0.326	0.051
高 (2.5-3.5 万)	2.659	0.569	439	0.337	0.048
很高 (>3.5 万)	2.636	0.574	694	0.348	0.048

第三问程序:

```
from typing import final
import numpy as np
import pandas as pd
from matplotlib.pyplot import title
from scipy.cluster.hierarchy import weighted
from scipy.stats import f_oneway, pearsonr
import warnings
warnings.filterwarnings("ignore")
import matplotlib.pyplot as plt
import scipy.stats as stars
import warnings
warnings.filterwarnings("ignore")
plt.rcParams["font.sans-serif"] = ["SimHei", "Arial Unicode
Ms", "DejaVu Sans"]
```



```

plt.rcParams["axes.unicode_minus"] = False

class TeachingAnalysis:
    """初始化分析参数"""
    # 高校教师属性分：性别、职称、年龄、学科、学科类型、学历、学校类型
    def __init__(self):
        self.genders = ["男", "女"]
        self.job_names = ["助教", "讲师", "副教授", "教授"]
        self.ages = ["25-35 岁", "36-45 岁", "46-55 岁", "56-65 岁"]
        self.majors = ["理工", "文学", "财经", "管理", "医学", "师范", "艺术", "体育"]
        self.subject_types = ["理论课程", "实验课程", "综合型课程"]
        self.education_levels = ["硕士", "博士", "院士"]
        self.universities_types = ["双一流学校", "重点学校", "普通本科学
        校", "高职学校"]

        self.direction_weights = {
            # 一级指标权重 （由问题 1 得出）
            "数字化知识和技能": 0.051510,
            "数字化教学设计": 0.175334,
            "数字化教学评价": 0.159995,
            "数字化教学实施": 0.114249,
            "数字社会责任": 0.252282,
            "数字化未来发展": 0.246630
        }

        self.quantity_directions = [
            # 人才指导指标
            "学生满意度",
            "课程效果",
            "创新能力",
            "实践能力",
            "就业能力",
            "综合素质"
        ]

    def generate_teacher_data(self, n_samples=1500):
        """
        生成教师的数据
        Args:
            n_samples (int): 样本数量

```

```

Returns:
    pd.DataFrame: 教师数据集
"""
# 教师基本属性
np.random.seed(42)
data = []
for i in range(n_samples):
    teacher_id = f"T{i+1:04d}"
    gender = np.random.choice(self.genders, p=[0.45, 0.55])
# 男女教师中女教师多
    age = np.random.randint(low=25, high=65)
    # 假定教龄为年龄减去 22 即 22 岁开始工作
    max_teaching_years = age - 22
    teaching_years =
np.random.randint(low=1, high=min(max_teaching_years + 1, 40))
    # 假定教师职称与年龄有关
    job_names = self._assign_title_by_experience(age,
teaching_years)
    # 学历分布
    education = np.random.choice(self.education_levels,
                                p=[0.37, 0.57, 0.06])
    # 专业分布
    major = np.random.choice(self.majors,
p=[0.3, 0.13, 0.12, 0.14, 0.1, 0.18, 0.03])
    # 授课类型
    subject_types = np.random.choice(self.subject_types,
                                p=[0.4, 0.25, 0.35])
    # 工作周长 / 学生数
    weekly_hours = np.random.randint(low=16, high=22)
    student_count = np.random.randint(low=40, high=200)
    # 科研项目数量 / 论文数量
    research_projects = np.random.poisson(2)
    publications = np.random.poisson(3)

    # 通过指导使教师拥有数字化能力
    teaching_hours = np.random.randint(low=20, high=150)

```

```

        tech_certifications = np.random.randint(low=0,high=5)

        # 依据教师属性来计算数字胜任力各指标分数
        digital_scores =
self._calculate_digital_competency_scores(

gender,major,age,job_names,education,subject_types,weekly_hours,

student_count,research_projects,teaching_hours,tech_certification
s

    )
    # 计算综合数字胜任力各指标分数
    comprehensive_score = sum([digital_scores[ind] * weight
                                for ind,weight in
self.direction_weights.items()])
    quality_scores = self._quality_calculate_scores(
        gender, age, teaching_years, job_names, education,
major, subject_types,
        weekly_hours, student_count, comprehensive_score
    )
    # 计算人才培养的质量指标分数
    quality_comprehensive =
np.mean(list(quality_scores.values()))

    data.append({
        "teacher_id": teacher_id,
        "gender": gender,
        "ages": age,
        "teaching_years": teaching_years,
        "job_names": job_names,
        "education": education,
        "major": major,
        "subject_types": subject_types,
        "weekly_hours": weekly_hours,
        "student_count": student_count,
        "research_projects": research_projects,
        "publications": publications,

```

```

        "teaching_hours": teaching_hours,
        "tech_certifications": tech_certifications,
        **digital_scores,
        "digital_comprehensive": round(comprehensive_score,
3),

        **quality_scores,
        "quality_comprehensive": round(quality_comprehensive,
3)

    })

    return pd.DataFrame(data)

    def _assign_title_by_experience(self, age, teaching_years):    #"
助教","讲师","副教授","教授"
        """根据年龄和教龄给予职称"""
        if age < 30 or teaching_years < 3:
            return np.random.choice(r=["助教","讲师"], p=[0.7,0.3])
        elif age <35 or teaching_years < 8:
            return np.random.choice(r=["讲师","副教授"], p=[0.8,0.2])
        elif age <45 or teaching_years < 15:
            return np.random.choice(r=["讲师","副教授","教授"],
p=[0.3,0.6,0.1])
        else:
            return np.random.choice(r=["副教授","教授"], p=[0.6,0.4])

    def _calculate_digital_competency_scores(self,
gender,age,teaching_years,
job_name,education,major,subject_type,
teaching_hours,tech_certifications):
        """计算数字胜任力各指标所得分数"""
        base_scores = {
            "数字化知识和技能": 2.7,
            "数字化教学设计": 3.0,
            "数字化教学评价": 3.2,
            "数字化教学实施": 2.8,
            "数字社会责任": 2.9,

```

```

        "数字化未来发展": 3.3
    }

    scores = {}

    for direction, base_score in base_scores.items():
        score = base_score

        # 年龄的影响
        age_factor = 1 + 0.3 * np.exp(-((age - 35) / 15) ** 2)

        # 教龄的影响
        exp_factor = 1 + 0.1 * min(teaching_years / 10, 1)

        # 不同职称的影响
        job_name_factors = {"助教": 0.9, "讲师": 1.0, "副教授": 1.1, "教授": 1.2}

        job_name_factor = job_name_factors[job_name]

        # 教师学历的影响
        education_factors = {"硕士": 1.0, "博士": 1.15, "院士": 1.2}
        education_factor = education_factors[education]

        # 专业的影响
        major_factors = {
            "理工": 1.2, "文学": 0.9, "财经": 1.0, "管理": 0.8,
            "医学": 1.2, "师范": 1.05, "艺术": 1.1, "体育": 0.8
        }

        major_factor = major_factors[major]

        # 授课类型的影响
        subject_type_factors = {"理论课程": 0.95, "实验课程": 1.1, "综合型课程": 1.05}

        subject_type_factor = subject_type_factors[subject_type]

        # 指导影响
        teaching_hours_factor = 1 + 0.002 * teaching_hours

        # 认证影响
        cert_factor = 1 + 0.05 * tech_certifications

        # 男女性别的影响
        gender_factor = 1.02 if gender == "女" else 1.0

        # 指标针对性调整
        if direction == "数字化知识和技能":
            teaching_hours_factor *= 1.2
            major_factor *= 1.1

        elif direction == "数字化教学实施":

```

```

        subject_type_factor *= 1.1
        age_factor *= 0.9
    elif direction == "数字化未来发展":
        job_name_factor *= 1.1
        education_factor *= 1.1
    # 计算综合分数
    final_score = (score * age_factor * exp_factor *
job_name_factor *
                    education_factor * major_factor *
subject_type_factor *
                    teaching_hours_factor * cert_factor *
gender_factor)
    # 添加随机性
    final_score *= np.random.normal(loc=0, scale=0.2)
    # 限制在 1~~5 分以内
    scores[direction] = np.clip(final_score, a_min=1, a_max=5)
    return scores

    def _calculate_quality_scares(self, gender, age,
teaching_years, job_name,
                                education, major, subject_type,
weekly_hours,
                                student_count, digital_score):
    """计算人才教导质量各指标所得分数"""
    scores = {}
    for direction in self.quantity_directions:
        base_score = 3.5
        digital_factor = 0.8 + 0.4 * (digital_score / 5.0)
        experience_factor = 1 + 0.2 * min(teaching_years / 15, 1)
        job_name_factors = {"助教": 0.9, "讲师": 1.0, "副教授": 1.1, "
教授": 1.15}
        job_name_factor = job_name_factors[job_name]
        workload_factor = 1 + 0.1 * np.exp(-(weekly_hours - 18)
/ 8) ** 2)
        if student_count < 50:
            size_factor = 1.1
        elif student_count < 100:
            size_factor = 0.9

```

```

        else:
            size_factor = 0.85
        if direction in ["学习研修", "研究创新"]:
            major_factors = {
                "理工": 1.3, "文学": 0.9, "财经": 0.9, "管理": 0.7,
                "医学": 1.3, "师范": 1.0, "艺术": 0.9, "体育": 0.7
            }
            major_factor = major_factors[major]
        else:
            major_factor = 1.0

        # 计算综合分数
        final_score = (base_score * digital_factor *
experience_factor *
                        job_name_factor * workload_factor *
size_factor * major_factor)
        # 添加随机性
        final_score *= np.random.normal(loc=0, scale=0.17)
        # 限制在 1~5 分以内
        scores[direction] = np.clip(final_score, a_min=1, a_max=5)
    return scores

def analyze_attribute_effects(self, data):
    """
    进一步分析教师属性对数字胜任力及教导质量的影响
    Args:
        data (pd.DataFrame): 教师数据
    Returns:
        dict: 分析结果
    """
    print("=" * 75)
    print("教师属性对数字胜任力及教导质量的影响分析")
    print("=" * 75)

    results = {}
    print("\n 性别影响分析")
    print("=" * 40)
    gender_results = self._analyze_gender_effects(data)

```

```

        results["gender"] = gender_results
        print("\n 年龄影响分析")
        print("=" * 40)
        age_results = self._analyze_age_effects(data)
        results["age"] = age_results
        print("\n 职称影响分析")
        print("=" * 40)
        job_name_results = self._analyze_job_name_effects(data)
        results["job_name"] = job_name_results
        print("\n 学历影响分析")
        print("=" * 40)
        education_results = self._analyze_education_effects(data)
        results["education"] = education_results
        print("\n 专业影响分析")
        print("=" * 40)
        major_results = self._analyze_major_effects(data)
        results["major"] = major_results
        print("\n 授课类型影响分析")
        print("=" * 40)
        subject_type_results =
self._analyze_subject_type_effects(data)
        results["subject_type"] = subject_type_results
        print("\n 构建综合模型分析")
        print("=" * 40)
        model_results = self._build_comprehensive_model(data)
        results["comprehensive_model"] = model_results
        print("\n 影响路径分析")
        print("=" * 40)
        path_results = self._path_analysis(data)
        results["path_analysis"] = path_results
        return results

def _analyze_gender_effects(self, data):
    """性别影响分析"""
    gender_stats = data.groupby("gender").agg({
        "digital_comprehensive": ["mean", "std", "count"],
        "quality_comprehensive": ["mean", "std", "count"]
    }).round(3)

```



```

print("性别统计: ")
print(gender_stats)

# t 检验
male_digital = data[data["gender"] == "男"]
male_digital["digital_comprehensive"]
female_digital = data[data["gender"] == "女"]
female_digital["digital_comprehensive"]
t_stat_digital, p_val_digital =
gender_stats.ttest_ind(male_digital, female_digital)

male_quality = data[data["gender"] == "男"]
male_quality["quality_comprehensive"]
female_quality = data[data["gender"] == "女"]
female_quality["quality_comprehensive"]
t_stat_quality, p_val_quality =
gender_stats.ttest_ind(male_quality, female_quality)

print(f"\n 数字胜任力差异性 t 检验:
t={t_stat_digital:.4f},p={p_val_digital:.4f}")
print(f"对教导质量的性别进行差异性 t 检验:
t={t_stat_quality:.4f},p={p_val_quality:.4f}")
return {
    "stats": gender_stats,
    "digital_t_test": {"t_stat": t_stat_digital, "p_value":
p_val_digital},
    "quality_t_test": {"t_stat": t_stat_quality, "p_value":
p_val_quality}
}

def _analyze_age_effects(self, data):
    """年龄影响分析"""
    data["age_group"] = pd.cut(data["age"],
                                bins=[25, 35, 45, 55, 65],
                                labels=["25-35 岁", "36-45 岁", "46-55 岁",
"56-65 岁"])
    age_stats = data.groupby("age_group").agg({
        "digital_comprehensive": ["mean", "std", "count"],

```

```

        "quality_comprehensive": ["mean", "std", "count"]
    }).round(3)
    print("年龄组统计: ")
    print(age_stats)
    # 相关性分析
    corr_digital, p_digital = pearsonr(data["age"],
data["digital_comprehensive"])
    corr_quality, p_quality = pearsonr(data["age"],
data["quality_comprehensive"])

    print(f"\n 年龄与数字胜任力相关性: r={corr_digital:.4f},
p={p_digital:.4f}")
    print(f"年龄与教导质量相关性: r={corr_quality:.4f},
p={p_quality:.4f}")

    return {
        "stats": age_stats,
        "digital_correlation": {"r": corr_digital, "p": p_digital},
        "quality_correlation": {"r": corr_quality, "p": p_quality}
    }

def _analyze_title_effects(self, data):
    """职称影响分析"""
    job_name_stats = data.groupby("job_name").agg({
        "digital_comprehensive": ["mean", "std", "count"],
        "quality_comprehensive": ["mean", "std", "count"]
    }).round(3)
    print("职称统计: ")
    print(job_name_stats)
    # 方差分析
    groups_digital = [data[data["job_name"] ==
t]["digital_comprehensive"].values
        for t in self.job_name]
    groups_quality = [data[data["job_name"] ==
t]["quality_comprehensive"].values
        for t in self.job_name]
    f_digital, p_digital = job_name_stats.f_oneway(*groups_digital)
    f_quality, p_quality = job_name_stats.f_oneway(*groups_quality)

```

```

        print(f"\n 数字胜任力职称影响 (ANOVA) : F={f_digital:.4f},
p={p_digital:.4f}")
        print(f"教导质量职称影响 (ANOVA) : F={f_quality:.4f},
p={p_quality:.4f}")
        return {
            "stats": job_name_stats,
            "digital_ANOVA": {"f_stat": f_digital, "p_value": p_digital},
            "quality_ANOVA": {"f_stat": f_quality, "p_value": p_quality}
        }
def _analyze_education_effects(self, data):
    """学历影响分析"""
    education_stats = data.groupby("education").agg({
        "digital_comprehensive": ["mean", "std", "count"],
        "quality_comprehensive": ["mean", "std", "count"]
    }).round(3)
    print("学历统计: ")
    print(education_stats)
    # 方差分析
    groups_digital = [data[data["education"] ==
t]["digital_comprehensive"].values
                        for t in self.education_levels]
    groups_quality = [data[data["education"] ==
t]["quality_comprehensive"].values
                        for t in self.education_levels]
    f_digital, p_digital = education_stats.f_oneway(*groups_digital)
    f_quality, p_quality = education_stats.f_oneway(*groups_quality)
    print(f"\n 数字胜任力学历影响 (ANOVA) : F={f_digital:.4f},
p={p_digital:.4f}")
    print(f"教导质量学历影响 (ANOVA) : F={f_quality:.4f},
p={p_quality:.4f}")
    return {
        "stats": education_stats,
        "digital_ANOVA": {"f_stat": f_digital, "p_value": p_digital},
        "quality_ANOVA": {"f_stat": f_quality, "p_value": p_quality}
    }
def _analyze_major_effects(self, data):
    """专业影响分析"""

```

```

major_stats = data.groupby("major").agg({
    "digital_comprehensive": ["mean", "std", "count"],
    "quality_comprehensive": ["mean", "std", "count"]
}).round(3)
print("专业统计: ")
print(major_stats)
# 方差分析
groups_digital = [data[data["major"] ==
m]["digital_comprehensive"].values
                    for m in self.majors]
groups_quality = [data[data["major"] ==
m]["quality_comprehensive"].values
                   for m in self.majors]
f_digital, p_digital = major_stats.f_oneway(*groups_digital)
f_quality, p_quality = major_stats.f_oneway(*groups_quality)
print(f"\n 数字胜任力专业影响 (ANOVA) : F={f_digital:.4f},
p={p_digital:.4f}")
print(f" 教导质量专业影响 (ANOVA) : F={f_quality:.4f},
p={p_quality:.4f}")
return {
    "stats": major_stats,
    "digital_ANOVA": {"f_stat": f_digital, "p_value": p_digital},
    "quality_ANOVA": {"f_stat": f_quality, "p_value": p_quality}
}

def _analyze_subject_type_effects(self, data):
    """授课类型影响分析"""
    subject_type_stats = data.groupby("subject_type").agg({
        "digital_comprehensive": ["mean", "std", "count"],
        "quality_comprehensive": ["mean", "std", "count"]
    }).round(3)
    print("授课类型统计: ")
    print(subject_type_stats)
    # 方差分析
    groups_digital = [data[data["subject_type"] ==
s]["digital_comprehensive"].values
                      for s in self.subject_types]
    groups_quality = [data[data["subject_type"] ==

```

```

s]["quality_comprehensive"].values
        for s in self.subject_types]
    f_digital, p_digital =
subject_type_stats.f_oneway(*groups_digital)
    f_quality, p_quality =
subject_type_stats.f_oneway(*groups_quality)
    print(f"\n 数字胜任力授课类型影响 (ANOVA) : F={f_digital:.4f},
p={p_digital:.4f}")
    print(f"教导质量授课类型影响 (ANOVA) : F={f_quality:.4f},
p={p_quality:.4f}")
    return {
        "stats": subject_type_stats,
        "digital_ANOVA": {"f_stat": f_digital, "p_value": p_digital},
        "quality_ANOVA": {"f_stat": f_quality, "p_value": p_quality}
    }
def _build_comprehensive_model(self, data):
    """构建综合模型分析"""
    digital_directions = list(self.direction_weights.keys())
    attribute_effects = {}
    for direction in digital_directions:
        direction_effects = {}

        gender_means = data.groupby("gender")[direction].mean()
        gender_effect = abs(gender_means["女"] - gender_means["男"])
        direction_effects["gender"] = gender_effect

        job_name_means = data.groupby("job_name")[direction].mean()
        job_name_effect = job_name_means.std()
        direction_effects["job_name"] = job_name_effect

        edu_means = data.groupby("education")[direction].mean()
        edu_effect = edu_means.std()
        direction_effects["education"] = edu_effect

        major_means = data.groupby("major")[direction].mean()
        major_effect = major_means.std()
        direction_effects["major"] = major_effect

```

```

        subject_means =
data.groupby("subject_type")[direction].mean()
        subject_effect = subject_means.std()
        direction_effects["subject_type"] = subject_effect

        age_corr = data[["age", direction]].corr().iloc[0,1]
        direction_effects["age"] = abs(age_corr)

        teaching_corr = data[["teaching_years",
direction]].corr().iloc[0, 1]
        direction_effects["teaching_years"] = abs(teaching_corr)

        attribute_effects[direction] = direction_effects

        comprehensive_effects = {}
        for attribute in
["gender", "job_name", "education", "major", "subject_type", "age", "te
aching_years"]:
            effect_sum = 0
            for direction, weight in self.direction_weights.items():
                effect_sum += attribute_effects[direction][attribute] *
weight
            comprehensive_effects[attribute] = effect_sum
        print("\n 各属性对数字胜任力指标的影响")
        for direction in digital_directions:
            print(f"\n {direction}指标: ")
            for attribute, effect in attribute_effects[direction].items():
                print(f"{attribute}: {effect:.4f}")
        print("\n 各属性对数字胜任力的综合影响: ")
        for attribute, effect in comprehensive_effects.items():
            print(f"{attribute}: {effect:.4f}")

        quality_effects = {}
        for direction in self.quality_directions:
            direction_effects = {}

```

```

gender_means = data.groupby("gender")[direction].mean()
gender_effect = abs(gender_means["女"] - gender_means["男"])
direction_effects["gender"] = gender_effect

job_name_means = data.groupby("job_name")[direction].mean()
job_name_effect = job_name_means.std()
direction_effects["job_name"] = job_name_effect

edu_means = data.groupby("education")[direction].mean()
edu_effect = edu_means.std()
direction_effects["education"] = edu_effect

major_means = data.groupby("major")[direction].mean()
major_effect = major_means.std()
direction_effects["major"] = major_effect

subject_means =
data.groupby("subject_type")[direction].mean()
subject_effect = subject_means.std()
direction_effects["subject_type"] = subject_effect

age_corr = data[["age", direction]].corr().iloc[0, 1]
direction_effects["age"] = abs(age_corr)

teaching_corr = data[["teaching_years",
direction]].corr().iloc[0, 1]
direction_effects["teaching_years"] = abs(teaching_corr)

quality_effects[direction] = direction_effects

quality_comprehensive_effects = {}
for attribute in ["gender", "job_name", "education", "major",
"subject_type", "age", "teaching_years"]:
    effect_sum = 0
    for direction, weight in self.quality_direction.items():
        effect_sum += quality_effects[direction][attribute]
    quality_comprehensive_effects[attribute] = effect_sum /

```

```

len(self.quality_directions)

print("\n 各属性对教导质量指标的影响")
for direction in self.quality_directions:
    print(f"\n {direction}指标: ")
    for attribute, effect in attribute_effects[direction].items():
        print(f"{attribute}: {effect:.4f}")
print("\n 各属性对教导质量的综合影响: ")
for attribute, effect in quality_comprehensive_effects.items():
    print(f"{attribute}: {effect:.4f}")
return {
    "digital_effects": attribute_effects,
    "digital_comprehensive": comprehensive_effects,
    "quality_effects": quality_effects,
    "quality_comprehensive": quality_comprehensive_effects
}

def _path_analysis(self, data):
    """影响路径分析"""
    features =
["age", "teaching_years", "weekly_hours", "student_count",
    "research_projects", "publications", "teaching_hours",
    "tech_certifications", "digital_comprehensive"]

    corr_matrix = data[features].corr()
    print("\n 特征相关矩阵: ")
    print(corr_matrix.round(4))

    quality_correlations = {}
    for direction in self.quality_directions:
        corr, p_value =
pearsonr(data["digital_comprehensive"], data[direction])
        quality_correlations[direction] = {
            "correlation": corr,
            "p_value": p_value
        }
        print(f"\n 数字胜任力与{direction}的相关性: ")
        print(f"相关系数: {corr: 4f}, p 值: {p_value:.4f}")
    return {

```



```

        "feature_correlations": corr_matrix,
        "quality_correlations": quality_correlations
    }
def main():
    analyzer = TeachingAnalysis()
    data = analyzer.generate_teacher_data(n_samples=1500)
    results = analyzer.analyze_attribute_effects(data)

    excel_data = {
        "性别影响分析": pd.DataFrame({
            "指标": ["数字胜任力", "教导质量"],
            "p 值": [results["gender"]["digital_t_test"]["p_value"],
                    results["gender"]["quality_t_test"]["p_value"]],
            "显著性": ["显著" if
results["gender"]["digital_t_test"]["p_value"] < 0.05 else "不显著",
                    "显著" if
results["gender"]["quality_t_test"]["p_value"] < 0.05 else "不显著"]
        }),
        "年龄影响分析": pd.DataFrame({
            "指标": ["数字胜任力", "教导质量"],
            "p 值": [results["age"]["digital_correlation"]["r"],
                    results["age"]["quality_correlation"]["r"]],
            "显著性": [results["age"]["digital_correlation"]["p"],
                    results["age"]["quality_correlation"]["p"]]
        }),
        "职称影响分析": pd.DataFrame({
            "指标": ["数字胜任力", "教导质量"],
            "F 值": [results["job_name"]["digital_ANOVA"]["f_stat"],
                    results["job_name"]["quality_ANOVA"]["f_stat"]],
            "p 值": [results["job_name"]["digital_ANOVA"]["p_value"],
                    results["job_name"]["quality_ANOVA"]["p_value"]],
            "显著性": ["显著" if
results["job_name"]["digital_ANOVA"]["p_value"] < 0.05 else "不显著",
                    "显著" if
results["job_name"]["quality_ANOVA"]["p_value"] < 0.05 else "不显著"]
    ]

```

```

    }),
    "学历影响分析": pd.DataFrame({
        "指标": ["数字胜任力", "教导质量"],
        "F 值": [results["education"]["digital_ANOVA"]["f_stat"],
                 results["education"]["quality_ANOVA"]["f_stat"]],
        "p 值": [results["education"]["digital_ANOVA"]["p_value"],
                 results["education"]["quality_ANOVA"]["p_value"]],
        "显著性": ["显著" if
results["education"]["digital_ANOVA"]["p_value"] < 0.05 else "不显著",
                 "显著" if
results["education"]["quality_ANOVA"]["p_value"] < 0.05 else "不显著"]
    }),
    "专业影响分析": pd.DataFrame({
        "指标": ["数字胜任力", "教导质量"],
        "F 值": [results["major"]["digital_ANOVA"]["f_stat"],
                 results["major"]["quality_ANOVA"]["f_stat"]],
        "p 值": [results["major"]["digital_ANOVA"]["p_value"],
                 results["major"]["quality_ANOVA"]["p_value"]],
        "显著性": ["显著" if
results["major"]["digital_ANOVA"]["p_value"] < 0.05 else "不显著",
                 "显著" if
results["major"]["quality_ANOVA"]["p_value"] < 0.05 else "不显著"]
    }),
    "授课类型的影响分析": pd.DataFrame({
        "指标": ["数字胜任力", "教导质量"],
        "F 值":
[results["subject_type"]["digital_ANOVA"]["f_stat"],
results["subject_type"]["quality_ANOVA"]["f_stat"]],
        "p 值":
[results["subject_type"]["digital_ANOVA"]["p_value"],
results["subject_type"]["quality_ANOVA"]["p_value"]],
        "显著性": ["显著" if
results["subject_type"]["digital_ANOVA"]["p_value"] < 0.05 else "

```

```

不显著",

                "显著" if
results["subject_type"]["quality_ANOVA"]["p_value"] < 0.05 else "
不显著"]
            )),
            "数字胜任力指标影响": pd.DataFrame(
                results["comprehensive_model"]["digital_effects"]
            ).T,
            "教导质量指标影响": pd.DataFrame(
                results["comprehensive_model"]["quality_effects"]
            ).T,
            "授课类型影响分析": pd.DataFrame({
                "属性":
list(results["comprehensive_model"]["digital_comprehensive"].keys
()),
                "数字胜任力影响":
list(results["comprehensive_model"]["digital_comprehensive"].valu
es()),
                "教导质量影响":
list(results["comprehensive_model"]["quality_comprehensive"].valu
es())
            )),
            "数字胜任力与教导质量相关性": pd.DataFrame({
                "指标": ["数字胜任力", "教导质量"],
                "F 值":
[results["subject_type"]["digital_ANOVA"]["f_stat"],
results["subject_type"]["quality_ANOVA"]["f_stat"]],
                "p 值":
[results["subject_type"]["digital_ANOVA"]["p_value"],
results["subject_type"]["quality_ANOVA"]["p_value"]],
                "显著性": ["显著" if
results["subject_type"]["digital_ANOVA"]["p_value"] < 0.05 else "
不显著",

                "显著" if
results["subject_type"]["quality_ANOVA"]["p_value"] < 0.05 else "

```

```
不显著"]
        })

    }

    print("分析完毕。")

if __name__ == "__main__":
    main()
```

（这里开始论文正文）

一级标题用四号黑体字并居中；二级、三级标题用小四号黑体字，左端对齐（不居中）。
论文中其他内容一律采用小四号宋体字，行距为 1.25 倍。

